asio异步读写操作及注意事项

Posted on 2023-04-05 19:45:48 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2O0QiktnG7ktdgARRndYzNg8R55)

简介

本文介绍boost asio的异步读写操作及注意事项，为保证知识便于读者吸收，仅介绍api使用的代码片段，下一节再编写完整的客户端和服务器程序。
所以我们定义一个session类，这个session类表示服务器处理客户端连接的管理类

```
1. class Session {
2. public:
3.     Session(std::shared_ptr<asio::ip::tcp::socket> socket);
4.     void Connect(const asio::ip::tcp::endpoint& ep);
5. private:
6.     std::shared_ptr<asio::ip::tcp::socket> _socket;
7. };
```

session类定义了一个socket成员变量，负责处理对端的连接读写，封装了Connect函数

```
1. void Session::Connect(const asio::ip::tcp::endpoint  &ep) {
2.     _socket->connect(ep);
3. }
```

这里只是简单意思一下，下面核心介绍异步读写api的使用

异步写操作

在写操作前，我们先封装一个Node结构，用来管理要发送和接收的数据，该结构包含数据域首地址，数据的总长度，以及已经处理的长度(已读的长度或者已写的长度)

```
1. //最大报文接收大小
2. const int RECVSIZE = 1024;
3. class  MsgNode {
4. public :
5.     MsgNode(const char* msg,  int total_len): _total_len(total_len), _cur_len(0){
6.         _msg = new char[total_len];
7.         memcpy(_msg, msg, total_len);
8.     }
9.
10.    MsgNode(int total_len) :_total_len(total_len), _cur_len(0) {
11.        _msg = new char[total_len];
12.    }
13.
14.    ~MsgNode(){
15.        delete[]_msg;
16.    }
17.    //消息首地址
18.    char* _msg;
19.    //总长度
20.    int _total_len;
21.    //当前长度
22.    int _cur_len;
23. };
```

写了两个构造函数，两个参数的负责构造写节点，一个参数的负责构造读节点。
接下来为Session添加异步写操作和负责发送写数据的节点

```
1. class Session{
2. public:
3.     void WriteCallBackErr(const boost::system::error_code & ec, std::size_t bytes_transferre
   d,
4.     std::shared_ptr<MsgNode>);
5.     void WriteToSocketErr(const std::string& buf);
6. private:
7.     std::shared_ptr<MsgNode> _send_node;
8. };
```

WriteToSocketErr函数为我们封装的写操作，WriteCallBackErr为异步写操作回调的函数，为什么会有三个参数呢，
我们可以看一下asio源码

```
template<typename WriteToken = BOOST_ASIO_DEFAULT_COMPLETION_TOKEN_TYPE(executor_type)>
BOOST_ASIO_INITFN_AUTO_RESULT_TYPE_PREFIX(WriteToken,
    void (boost::system::error_code, std::size_t))
async_write_some(const ConstBufferSequence& buffers,
    BOOST_ASIO_MOVE_ARG(WriteToken)token
    BOOST_ASIO_DEFAULT_COMPLETION_TOKEN(executor_type))
```

async_write_some是异步写的函数，这个异步写函数有两个参数，第一个参数为ConstBufferSequence常引用类型的buffers，
第二个参数为WriteToken类型，而WriteToken在上面定义了，是一个函数对象类型，返回值为void，参数为error_code和size_t，
所以我们为了调用async_write_some函数也要传入一个符合WriteToken定义的函数，就是我们声明的WriteCallBackErr函数，
前两个参数为WriteToken规定的参数，第三个参数为MsgNode的智能指针，这样通过智能指针保证我们发送的Node生命周期延长。
我们看一下WriteToSocketErr函数的具体实现

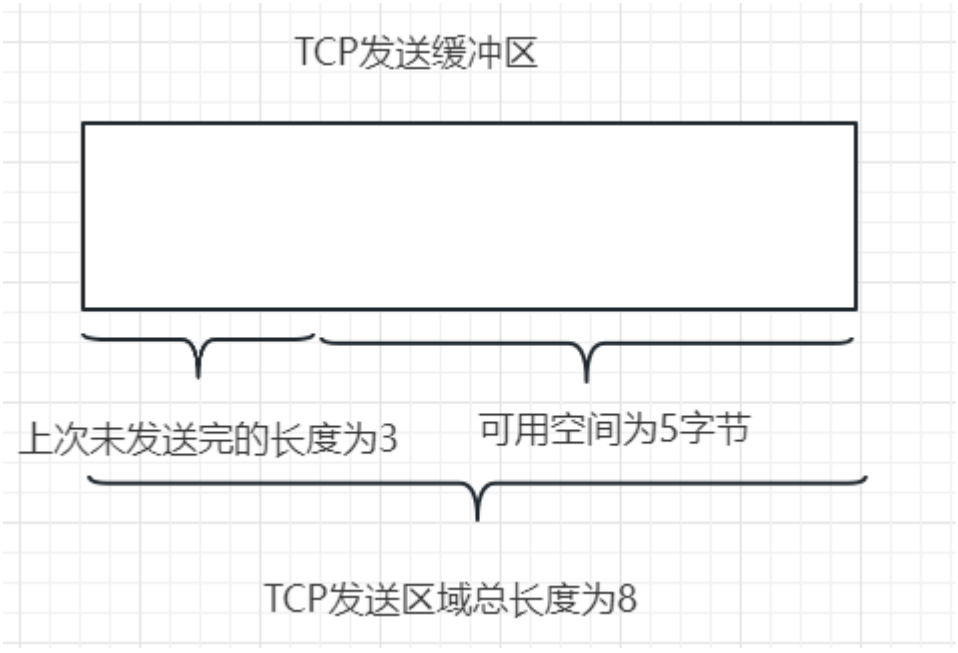
```
void Session::WriteToSocketErr(const std::string& buf) {
    _send_node = make_shared<MsgNode>(buf.c_str(), buf.length());
    //异步发送数据，因为异步所以不会一下发送完
    this->_socket->async_write_some(asio::buffer(_send_node->_msg,
        _send_node->_total_len),
        std::bind(&Session::WriteCallBackErr,
            this, std::placeholders::_1, std::placeholders::_2, _send_node));
}
```

因为WriteCallBackErr函数为三个参数且为成员函数，而async_write_some需要的回调函数为两个参数，所以我们通过bind将三个参数转换为两个参数的普通函数。
我们看看回调函数的实现

```
1. void Session::WriteCallBackErr(const boost::system::error_code& ec,
2.     std::size_t bytes_transferred, std::shared_ptr<MsgNode> msg_node) {
3.     if (bytes_transferred + msg_node->_cur_len
4.         < msg_node->_total_len) {
5.         _send_node->_cur_len += bytes_transferred;
6.         this->_socket->async_write_some(asio::buffer(_send_node->_msg+_send_node->_cur_le
   n,
7.             _send_node->_total_len-_send_node->_cur_len),
8.             std::bind(&Session::WriteCallBackErr,
9.                 this, std::placeholders::_1, std::placeholders::_2, _send_node));
10.    }
11. }
```

在WriteCallBackErr函数里判断如果已经发送的字节数没有达到要发送的总字节数，那么久更新节点已经发送的长度，然后计算剩余要发送的长度，如果有数据未发送完，再次调用async_write_some函数异步发送。

但是这个函数并不能投入实际应用，因为async_write_some回调函数返回已发送的字节数可能并不是全部长度。比如TCP发送缓存区总大小为8字节，但是有3字节未发送(上一次未发送完)，这样剩余空间为5字节



此时我们调用async_write_some发送hello world!实际发送的长度就是为5，也就是只发送了hello，剩余world!通过我们的回调继续发送。

而实际开发的场景用户是不清楚底层tcp的多路复用调用情况的，用户想发送数据的时候就调用WriteToSocketErr,或者循环调用WriteToSocketErr，很可能在一次没发送完数据还未调用回调函数时再次调用WriteToSocketErr，因为boost::asio封装的时epoll和iocp等多路复用模型，当写事件就绪后就发数据，发送的数据按照async_write_some调用的顺序发送，所以回

调函数内调用的async_write_some可能并没有被及时调用。
比如我们如下代码

```
1. //用户发送数据
2. WriteToSocketErr("Hello World!");
3. //用户无感知下层调用情况又一次发送了数据
4. WriteToSocketErr("Hello World!");
```

那么很可能第一次只发送了Hello，后面的数据没发完，第二次发送了Hello World!之后又发送了World!
所以对端收到的数据很可能是”HelloHello World! World!”
那怎么解决这个问题呢，我们可以通过队列保证应用层的发送顺序。我们在Session中定义一个发送队列，然后重新定义正确的异步发送函数和回调处理

```
1. class Session{
2. public:
3.     void WriteCallBack(const boost::system::error_code& ec, std::size_t bytes_transferred);
4.     void WriteToSocket(const std::string &buf);
5. private:
6.     std::queue<std::shared_ptr<MsgNode>> _send_queue;
7.     std::shared_ptr<asio::ip::tcp::socket> _socket;
8.     bool _send_pending;
9. };
```

定义了bool变量_send_pending，该变量为true表示一个节点还未发送完。
_send_queue用来缓存要发送的消息节点，是一个队列。
我们实现异步发送功能

```
void Session::WriteToSocket(const std::string& buf){
    //插入发送队列
    _send_queue.emplace(new MsgNode(buf.c_str(), buf.length()));
    //pending状态说明上一次有未发送完的数据
    if (_send_pending) {
        return;
    }
    //异步发送数据，因为异步所以不会一下发送完
    this->_socket->async_write_some(asio::buffer(buf), std::bind(&Session::WriteCallBack, this, std::placeholders::_1, std::placeholders::_2));
    _send_pending = true;
}

void Session::WriteCallBack(const boost::system::error_code & ec,  std::size_t bytes_transferred){
    if (ec.value() != 0) {
        std::cout << "Error , code is " << ec.value() << " . Message is " << ec.message();
        return;
    }
    //取出队首元素即当前未发送完数据
    auto & send_data = _send_queue.front();
    send_data->_cur_len += bytes_transferred;
    //数据未发送完，  则继续发送
    if (send_data->_cur_len < send_data->_total_len) {
        this->_socket->async_write_some(asio::buffer(send_data->_msg + send_data->_cur_len, send_data->_total_len-send_data->_cur_len),
            std::bind(&Session::WriteCallBack,
            this, std::placeholders::_1, std::placeholders::_2));
        return;
    }

    //如果发送完，则pop出队首元素
    _send_queue.pop();

    //如果队列为空，则说明所有数据都发送完,将pending设置为false
    if (_send_queue.empty()) {
        _send_pending = false;
    }
    //如果队列不是空，则继续将队首元素发送
    if (!_send_queue.empty()) {
        auto& send_data = _send_queue.front();
        this->_socket->async_write_some(asio::buffer(send_data->_msg + send_data->_cur_len,
```

```
    send_data->_total_len - send_data->_cur_len),
40.             std::bind(&Session::WriteCallBack,
41.                 this, std::placeholders::_1, std::placeholders::_2));
42.     }
43. }
```

async_write_some函数不能保证每次回调函数触发时发送的长度为要总长度，这样我们每次都要在回调函数判断发送数据是否完成，asio提供了一个更简单的发送函数async_send，这个函数在发送的长度未达到我们要求的长度时就不会触发回调，所以触发回调函数时要么时发送出错了要么是发送完成了,其内部的实现原理就是帮我们不断的调用async_write_some直到完成发送，所以async_send不能和async_write_some混合使用，我们基于async_send封装另外一个发送函数

```
1. //不能与async_write_some混合使用
2. void Session::WriteAllToSocket(const std::string& buf) {
3.     //插入发送队列
4.     _send_queue.emplace(new MsgNode(buf.c_str(), buf.length()));
5.     //pending状态说明上一次有未发送完的数据
6.     if (_send_pending) {
7.         return;
8.     }
9.     //异步发送数据，因为异步所以不会一下发送完
10.    this->_socket->async_send(asio::buffer(buf),
11.        std::bind(&Session::WriteAllCallBack, this,
12.            std::placeholders::_1, std::placeholders::_2));
13.    _send_pending = true;
14. }
15.
16. void Session::WriteAllCallBack(const boost::system::error_code& ec, std::size_t bytes_transf
    erred){
17.    if (ec.value() != 0) {
18.        std::cout << "Error occured! Error code = "
19.            << ec.value()
20.            << ". Message: " << ec.message();
21.
22.        return;
23.    }
24.
25.    //如果发送完，则pop出队首元素
26.    _send_queue.pop();
27.
28.    //如果队列为空，则说明所有数据都发送完,将pending设置为false
29.    if (_send_queue.empty()) {
30.        _send_pending = false;
31.    }
32.    //如果队列不是空，则继续将队首元素发送
33.    if (!_send_queue.empty()) {
34.        auto& send_data = _send_queue.front();
35.        this->_socket->async_send(asio::buffer(send_data->_msg + send_data->_cur_len, send_d
    ata->_total_len - send_data->_cur_len),
36.            std::bind(&Session::WriteAllCallBack,
37.                this, std::placeholders::_1, std::placeholders::_2));
38.    }
39. }
```

异步读操作

接下来介绍异步读操作，异步读操作和异步的写操作类似同样又async_read_some和async_receive函数，前者触发的回调函数获取的读数据的长度可能会小于要求读取的总长度，后者触发的回调函数读取的数据长度等于读取的总长度。
先基于async_read_some封装一个读取的函数ReadFromSocket，同样在Session类的声明中添加一些变量

```
1. class Session {
2. public:
3.     void ReadFromSocket();
4.     void ReadCallBack(const boost::system::error_code& ec, std::size_t bytes_transferred);
5. private:
6.     std::shared_ptr<asio::ip::tcp::socket> _socket;
7.     std::shared_ptr<MsgNode> _recv_node;
8.     bool _recv_pending;
9. };
```

_recv_node用来缓存接收的数据，_recv_pending为true表示节点正在接收数据，还未接受完。

```
// 不考虑粘包情况， 先用固定的字节接收
void Session::ReadFromSocket() {
    if (_recv_pending) {
        return;
    }
    // 可以调用构造函数直接构造，但不可用已经构造好的智能指针赋值
    /*auto _recv_nodez = std::make_unique<MsgNode>(RECVSIZE);
    _recv_node = _recv_nodez;*/
    _recv_node = std::make_shared<MsgNode>(RECVSIZE);
    _socket->async_read_some(asio::buffer(_recv_node->_msg, _recv_node->_total_len), std::bind(&Session::ReadCallBack, this,
        std::placeholders::_1, std::placeholders::_2));
    _recv_pending = true;
}

void Session::ReadCallBack(const boost::system::error_code& ec, std::size_t bytes_transferred){
    _recv_node->_cur_len += bytes_transferred;
    // 没读完继续读
    if (_recv_node->_cur_len < _recv_node->_total_len) {
        _socket->async_read_some(asio::buffer(_recv_node->_msg+_recv_node->_cur_len,
            _recv_node->_total_len - _recv_node->_cur_len), std::bind(&Session::ReadCallBack, this,
            std::placeholders::_1, std::placeholders::_2));
        return;
    }
    // 将数据投递到队列里交给逻辑线程处理，此处略去
    // 如果读完了则将标记置为false
    _recv_pending = false;
    // 指针置空
    _recv_node = nullptr;
}
```

我们基于async_receive再封装一个接收数据的函数

```
1. void Session::ReadAllFromSocket(const std::string& buf) {
2.     if (_recv_pending) {
3.         return;
4.     }
5.     //可以调用构造函数直接构造，但不可用已经构造好的智能指针赋值
6.     /*auto _recv_nodez = std::make_unique<MsgNode>(RECVSIZE);
7.     _recv_node = _recv_nodez;*/
8.     _recv_node = std::make_shared<MsgNode>(RECVSIZE);
9.     _socket->async_receive(asio::buffer(_recv_node->_msg, _recv_node->_total_len), std::bind
   (&Session::ReadAllCallBack, this,
10.         std::placeholders::_1, std::placeholders::_2));
11.     _recv_pending = true;
12. }
13.
14. void Session::ReadAllCallBack(const boost::system::error_code& ec, std::size_t bytes_transf
   erred) {
15.     _recv_node->_cur_len += bytes_transferred;
16.     //将数据投递到队列里交给逻辑线程处理，此处略去
17.     //如果读完了则将标记置为false
18.     _recv_pending = false;
19.     //指针置空
20.     _recv_node = nullptr;
21. }
```

同样async_read_some和async_receive不能混合使用，否则会出现逻辑问题。

源码连接

本文介绍了boost asio异步读写的操作，仅仅是代码片段和api的封装便于大家理解，下一篇利用这些异步api写一个异步的服务器展示收发效果。
源码连接
https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(5272) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(564) (JavaScript:void(0);)

热门评论



请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)